OOP II & Data Structures

Bit by Bit Coding - Term 2 | Week 8

Key Concepts

- Inheritance: a child class reuses code from a parent class.
- super() calls the parent's methods (e.g., its __init__).
- Method overriding: a child redefines a parent method with new behaviour.
- Polymorphism: different classes can be used the same way (same method name).
- Encapsulation: bundle data + behaviour; control access to internals.
- Data structures: linked list, binary tree, stack, queue.

Syntax Reference

Inheritance & super()

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):              # Dog inherits Animal
    def __init__(self, name):
        super().__init__(name)  # call parent constructor
    def speak(self):            # override
        return "Woof"

d = Dog("Rex")
print(d.name, d.speak())       # Rex Woof
```

Linked List (Node class)

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

head = Node(1)
head.next = Node(2)
head.next.next = Node(3)

cur = head                     # traverse
while cur:
    print(cur.value)
    cur = cur.next
```

Binary Tree

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert(node, value):
    if node is None:
        return TreeNode(value)
    if value < node.value:
        node.left = insert(node.left, value)
    else:
        node.right = insert(node.right, value)
    return node

def inorder(node):
    if node:
        inorder(node.left)
        print(node.value)
        inorder(node.right)
```

Stack & Queue

```
stack = []            # LIFO: last in, first out
stack.append(1); stack.append(2)
stack.pop()           # 2

from collections import deque
queue = deque()       # FIFO: first in, first out
queue.append(1); queue.append(2)
queue.popleft()       # 1
```

Common Patterns

- Use inheritance to share common behaviour; override only what changes.
- Linked list: each Node holds a value + a link to the next Node.
- Tree: insert by comparing left/right; traverse with recursion.

Tips & Common Mistakes

- Call super().__init__(...) in the child to set up parent attributes.
- Forgetting to return from recursive tree insert breaks the links.
- Stack = LIFO (pop); Queue = FIFO (popleft). Don't mix them up.
- A None next/child pointer marks the end - always check before using it.